

1 — Prototype: own first, then inherited

OBJECTS LOOKUP INHERITANCE

Q. You never added `publish` to `piece`, so where does `piece.publish()` come from?

ANSWER. Every object carries a hidden link to another object called its prototype. When you read a property, JavaScript looks on the object itself first; if the property is not there, it follows that hidden link up to the prototype and uses whatever it finds. `publish` was never put on `piece`, so the lookup walks up to `Article.prototype` and finds it there.

WHY IT WORKS. A property stored directly on the object is an own property, returned at once. A property found by walking up the chain is inherited. There is only one shared copy of `publish` on `Article.prototype`, but every instance can reach it, so a million `Article` objects share one method instead of each carrying its own.

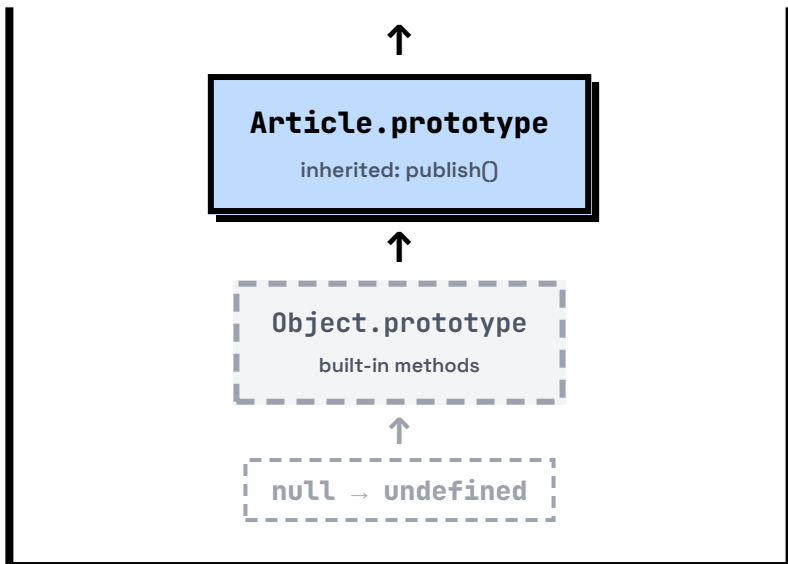
examples/Article.js

```
class Article {  
  publish() { return "published"; } // on  
  Article.prototype  
}  
const piece = new Article(); // a new  
instance  
piece.headline = "Markets rally"; // an own  
property  
console.log(piece.headline); // "Markets  
rally" – own  
console.log(piece.publish()); // "published"  
– inherited  
console.log(piece.author); // undefined –  
not on the chain
```

THE LOOKUP WALKS UP

piece

own: headline = "Markets rally"



- ① **Summary.** A read checks own properties first, then walks up the prototype chain until it finds a match or hits `null`. Today a read only *finds* a value; the next idea turns a read into something that can *run* code.